Enrollment No:-240840116014

Practical 2:-Perform data inspection, cleaning, handling missing values, duplicate removal, and data transformation using Pandas.

Step 1: Import Pandas and Load the Dataset

```
# Import the pandas library
import pandas as pd
```

Step 2: Create Dataset

```
# Create a dictionary
data = {
    "ID": [101, 102, 103, 104, 105, 106, 107, 108],
    "Name": ["Rahul", "", "Priya", "Amit", "None", "Riya", "Ajay", "Naresh"],
    "Age": [20, 21, None, 20, 20, 22, 23, 28],
    "City": ["Surat", "Navsari", "Surat", "Bharuch", "Bharuch", "Navsari", "Baroda", "Ahmedabad"],
    "Marks": [85, None, 78, 88, 88, 81, 90, 93]
}
```

```
# Create DataFrame
df = pd.DataFrame(data)

# Display DataFrame
print(df)
```

```
    ID    Name   Age       City  Marks
0  101   Rahul  20.0      Surat   85.0
1  102          21.0    Navsari    NaN
2  103   Priya   NaN      Surat   78.0
3  104    Amit  20.0    Bharuch   88.0
4  105    None  20.0    Bharuch   88.0
5  106    Riya  22.0    Navsari   81.0
6  107    Ajay  23.0     Baroda   90.0
7  108  Naresh  28.0  Ahmedabad   93.0
```

Step 3: Data inspection

```
#Display first 5 Records of dataset
df.head()
```

	ID	Name	Age	City	Marks
0	101	Rahul	20.0	Surat	85.0
1	102		21.0	Navsari	NaN
2	103	Priya	NaN	Surat	78.0
3	104	Amit	20.0	Bharuch	88.0
4	105	None	20.0	Bharuch	88.0

Next steps: Generate code with df | New interactive sheet

```
#Display last 5 Records of dataset
df.tail()
```

	ID	Name	Age	City	Marks
3	104	Amit	20.0	Bharuch	88.0
4	105	None	20.0	Bharuch	88.0
5	106	Riya	22.0	Navsari	81.0
6	107	Ajay	23.0	Baroda	90.0
7	108	Naresh	28.0	Ahmedabad	93.0

```
# Display detailed information of the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 5 columns):
```

```
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   ID      8 non-null      int64
 1   Name    8 non-null      object
 2   Age     7 non-null      float64
 3   City    8 non-null      object
 4   Marks   7 non-null      float64
dtypes: float64(2), int64(1), object(2)
memory usage: 452.0+ bytes
```

```
#statistical summary of numerical columns
df.describe()
```

	ID	Age	Marks
count	8.00000	7.000000	7.000000
mean	104.50000	22.000000	86.142857
std	2.44949	2.886751	5.209881
min	101.00000	20.000000	78.000000
25%	102.75000	20.000000	83.000000
50%	104.50000	21.000000	88.000000
75%	106.25000	22.500000	89.000000
max	108.00000	28.000000	93.000000

```
#Display dimension of dataset
df.shape
```

```
(8, 5)
```

```
# Display all column names
df.columns
```

```
Index(['ID', 'Name', 'Age', 'City', 'Marks'], dtype='object')
```

```
# Display data types of each column
df.dtypes
```

	0
ID	int64
Name	object
Age	float64
City	object
Marks	float64

dtype: object

Step 4: Check Missing Values

```
#Check for missing values in each column
df.isnull().sum()
```

	0
ID	0
Name	0
Age	1
City	0
Marks	1

dtype: int64

Step 5: Replace missing values

```
#Replace missing values in the marks column
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
```

```
#Display updated Dataset
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20.0	Surat	85.000000
1	102		21.0	Navsari	86.142857
2	103	Priya	NaN	Surat	78.000000
3	104	Amit	20.0	Bharuch	88.000000
4	105	None	20.0	Bharuch	88.000000
5	106	Riya	22.0	Navsari	81.000000
6	107	Ajay	23.0	Baroda	90.000000
7	108	Naresh	28.0	Ahmedabad	93.000000

Next steps: Generate code with df | New interactive sheet

```
#Replace missing values in the age column
df["Age"] = df["Age"].fillna(df["Age"].mean())
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20.0	Surat	85.000000
1	102		21.0	Navsari	86.142857
2	103	Priya	22.0	Surat	78.000000
3	104	Amit	20.0	Bharuch	88.000000
4	105	None	20.0	Bharuch	88.000000
5	106	Riya	22.0	Navsari	81.000000
6	107	Ajay	23.0	Baroda	90.000000
7	108	Naresh	28.0	Ahmedabad	93.000000

Next steps: Generate code with df | New interactive sheet

```
#Replace missing names with text "unknown"
df["Name"]= df["Name"].fillna("unknowm")
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20.0	Surat	85.000000
1	102		21.0	Navsari	86.142857
2	103	Priya	22.0	Surat	78.000000
3	104	Amit	20.0	Bharuch	88.000000
4	105	None	20.0	Bharuch	88.000000
5	106	Riya	22.0	Navsari	81.000000
6	107	Ajay	23.0	Baroda	90.000000
7	108	Naresh	28.0	Ahmedabad	93.000000

Next steps: Generate code with df | New interactive sheet

```
#display updated column
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20.0	Surat	85.000000
1	102		21.0	Navsari	86.142857
2	103	Priya	22.0	Surat	78.000000
3	104	Amit	20.0	Bharuch	88.000000
4	105	None	20.0	Bharuch	88.000000
5	106	Riya	22.0	Navsari	81.000000
6	107	Ajay	23.0	Baroda	90.000000
7	108	Naresh	28.0	Ahmedabad	93.000000

Next steps: Generate code with df | New interactive sheet

Step 6: Detect duplicate data

```
#check whether row is duplicated
df.duplicated()
```

	0
0	False
1	False
2	False
3	False
4	False
5	False
6	False
7	False

dtype: bool

```
#Count the total number of duplicate row
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20.0	Surat	85.000000
1	102		21.0	Navsari	86.142857
2	103	Priya	22.0	Surat	78.000000
3	104	Amit	20.0	Bharuch	88.000000
4	105	None	20.0	Bharuch	88.000000
5	106	Riya	22.0	Navsari	81.000000
6	107	Ajay	23.0	Baroda	90.000000
7	108	Naresh	28.0	Ahmedabad	93.000000

Next steps: Generate code with df | New interactive sheet

Step 7: Remove duplicated records

```
#Remove duplicated rows
df = df.drop_duplicates()
df
```

	ID	Name	Age	City	Marks
0	101	Rahul	20.0	Surat	85.000000
1	102		21.0	Navsari	86.142857
2	103	Priya	22.0	Surat	78.000000
3	104	Amit	20.0	Bharuch	88.000000
4	105	None	20.0	Bharuch	88.000000
5	106	Riya	22.0	Navsari	81.000000
6	107	Ajay	23.0	Baroda	90.000000
7	108	Naresh	28.0	Ahmedabad	93.000000

Next steps: Generate code with df | New interactive sheet

Step 8: Rename a column

```
#Rename the column "Marks" to "Score"
df.rename(columns = {"Marks" : "Score"}, inplace = True)
df
```

	ID	Name	Age	City	Score
0	101	Rahul	20.0	Surat	85.000000
1	102		21.0	Navsari	86.142857
2	103	Priya	22.0	Surat	78.000000
3	104	Amit	20.0	Bharuch	88.000000
4	105	None	20.0	Bharuch	88.000000
5	106	Riya	22.0	Navsari	81.000000
6	107	Ajay	23.0	Baroda	90.000000
7	108	Naresh	28.0	Ahmedabad	93.000000

Next steps: Generate code with df | New interactive sheet

Step 9: Change Data type

```
#convert the age column from float tp integer
df["Age"]=df["Age"].astype(int)
df.dtypes
```

	0
ID	int64
Name	object
Age	int64
City	object
Score	float64

dtype: object

Step 10:Create a new column

```
#Create a new column "Final Score"
df["Final Score"]=df["Score"] = 5
df
```

	ID	Name	Age	City	Score	Final Score
0	101	Rahul	20	Surat	5	5
1	102		21	Navsari	5	5
2	103	Priya	22	Surat	5	5
3	104	Amit	20	Bharuch	5	5
4	105	None	20	Bharuch	5	5
5	106	Riya	22	Navsari	5	5
6	107	Ajay	23	Baroda	5	5
7	108	Naresh	28	Ahmedabad	5	5

Next steps: Generate code with df | New interactive sheet

Step 11:Convert a Student Name to Uppercase

```
#Convert all Students names to Uppercase
df["Name"] = df["Name"].str.upper()
df
```

	ID	Name	Age	City	Score	Final Score
0	101	RAHUL	20	Surat	5	5
1	102		21	Navsari	5	5
2	103	PRIYA	22	Surat	5	5
3	104	AMIT	20	Bharuch	5	5
4	105	NONE	20	Bharuch	5	5
5	106	RIYA	22	Navsari	5	5
6	107	AJAY	23	Baroda	5	5
7	108	NARESH	28	Ahmedabad	5	5

Next steps: Generate code with df | New interactive sheet

Step 12:Convert to Lowercase

```
#Convert City Names to lowercase
df["City"] = df["City"].str.lower()
df
```

	ID	Name	Age	City	Score	Final Score
0	101	RAHUL	20	surat	5	5
1	102		21	navsari	5	5
2	103	PRIYA	22	surat	5	5
3	104	AMIT	20	bharuch	5	5
4	105	NONE	20	bharuch	5	5
5	106	RIYA	22	navsari	5	5
6	107	AJAY	23	baroda	5	5
7	108	NARESH	28	ahmedabad	5	5

Next steps: Generate code with df | New interactive sheet

Step 13: Remove extra spaces

```
#Remove leading and trailing spaces from city names
df["Ciyy"] = df["City"].str.strip()
df
```

	ID	Name	Age	City	Score	Final Score	Ciyy
0	101	RAHUL	20	surat	5	5	surat
1	102		21	navsari	5	5	navsari
2	103	PRIYA	22	surat	5	5	surat
3	104	AMIT	20	bharuch	5	5	bharuch
4	105	NONE	20	bharuch	5	5	bharuch
5	106	RIYA	22	navsari	5	5	navsari
6	107	AJAY	23	baroda	5	5	baroda
7	108	NARESH	28	ahmedabad	5	5	ahmedabad

Next steps: Generate code with df | New interactive sheet

Step 14: Sort the dataset

```
# Step 14: Sort the dataset
df = df.sort_values(by="Final Score", ascending=False)

# Print the complete dataset
print(df.to_string(index=False))
```

```
 ID    Name  Age      City  Score  Final Score      Ciyy
101   RAHUL   20     surat      5            5     surat
102            21   navsari      5            5   navsari
103   PRIYA   22     surat      5            5     surat
104    AMIT   20   bharuch      5            5   bharuch
105    NONE   20   bharuch      5            5   bharuch
106    RIYA   22   navsari      5            5   navsari
107    AJAY   23    baroda      5            5    baroda
108 NARESH   28 ahmedabad      5            5 ahmedabad
```

Step 15: Save the cleaned Dataset